

Design and FPGA Implementation of a 5-Stage Pipelined RISC-V Processor with Software-Managed Sv32 TLB and Performance Analysis

Ved Patel

Roll No: B24503

SCEE — School of Computing and Electrical Engineering

Course Project — VL326: Fundamentals of Processor Architecture and SoC Design

Zybo Z7-10 (xc7z010clg400-1)

Abstract—This paper presents the design, verification, and FPGA deployment of a 5-stage in-order pipelined processor implementing the RISC-V RV32I base integer instruction set with Zicsr extension, augmented with a software-managed 4-entry fully-associative Translation Lookaside Buffer (TLB) using Sv32-style addressing and FIFO replacement. The processor features full data forwarding, load-use hazard detection, and a complete trap handling infrastructure supporting page fault exceptions with instruction replay. Empirical performance analysis across four test configurations with varying working-set sizes (1–16 pages) demonstrates a dramatic TLB capacity cliff: address-translation overhead rises from 39–42% (cold-start misses only) to 87–88% of total execution time under thrashing conditions, representing an $8\times$ slowdown relative to an equivalent base CPU without virtual memory. Note that conventional CPI is *not* a meaningful metric in this trap-replay architecture, since each TLB miss inflates both the cycle and instruction counts. The design is synthesized and deployed on a Digilent Zybo Z7-10 FPGA at 62.5 MHz, with performance data captured via Xilinx Integrated Logic Analyzer (ILA).

Index Terms—RISC-V, pipelined processor, TLB, virtual memory, Sv32, FPGA, page fault, FIFO replacement, ILA, performance analysis

- A 4-entry fully-associative TLB with FIFO replacement policy
- A software-managed page fault handler implementing 2-level Sv32 page table walks
- Performance counters for cycles, instructions, stalls, and TLB statistics

Design scope. This implementation makes three deliberate pedagogical simplifications relative to the full RISC-V Sv32 specification: (1) only data memory accesses are translated—there is no instruction-side TLB; (2) R/W/X/U permission bits from page table entries are not stored or enforced; and (3) SATP is used purely as a base address register without MODE or ASID fields. These choices reduce hardware complexity while preserving the phenomena under study (TLB capacity effects and software page-walk overhead). They are explicitly documented as limitations in Section X.

The design targets the Digilent Zybo Z7-10 FPGA board (Xilinx Zynq-7010) and uses Vivado 2025.2 for synthesis, implementation, and debugging via the Xilinx Integrated Logic Analyzer (ILA).

I. INTRODUCTION

Virtual memory is a foundational abstraction in modern computer systems, enabling process isolation, flexible memory management, and the illusion of a large, contiguous address space. The Translation Lookaside Buffer (TLB) is a critical performance component that caches recent virtual-to-physical address translations to avoid costly page table walks on every memory access.

This project investigates the following research question: *What is the effective address-translation overhead as a fraction of total execution time when TLB coverage is exceeded?*

To answer this, we designed and implemented a complete RV32I pipelined processor with:

- A 5-stage in-order pipeline (IF, ID, EX, MEM, WB) with full data forwarding and load-use hazard stalling
- Zicsr extension for Control and Status Register operations

II. ARCHITECTURE OVERVIEW

A. Pipeline Design

The processor implements a classic 5-stage in-order pipeline:

- 1) **Instruction Fetch (IF):** Priority-based PC mux supporting trap, MRET, branch, stall, and sequential paths. Fetches instructions from 16 KB BRAM.
- 2) **Instruction Decode (ID):** Decodes all 40 RV32I instructions plus 6 Zicsr instructions (CSRRW/RS/RC and immediate variants), ECALL, EBREAK, and MRET. Generates immediate values for all 5 formats (I/S/B/U/J).
- 3) **Execute (EX):** ALU operations (11 operations including PASS for LUI), branch resolution, misalignment detection, trap detection (6 cause codes), and CSR data computation.
- 4) **Memory (MEM):** TLB-based address translation, store data alignment with byte-enable masks, load

data formatting (sign/zero extension for LB/LH/LW/LBU/LHU), and page fault generation.

- 5) **Writeback (WB):** 4-way result mux selecting from ALU, memory load, PC+4 (JAL/JALR), or CSR read data.

B. Data Hazard Management

The processor implements two-level data forwarding:

- **EX→EX forwarding:** Bypasses results from the EX/MEM register to the current EX stage (excludes load instructions).
- **MEM→EX forwarding:** Bypasses results from the MEM/WB register, including load data.

Load-use hazards are detected by the hazard unit, which inserts a 1-cycle pipeline bubble when an instruction in ID depends on a load currently in EX.

C. Key Design Parameters

Table I summarizes the processor’s design parameters.

TABLE I
PROCESSOR DESIGN PARAMETERS

Parameter	Value
ISA	RV32I + Zicsr
Pipeline depth	5 stages
Data forwarding	EX→EX and MEM→EX
Branch resolution	EX stage (2-cycle penalty)
Branch prediction	Static not-taken
TLB	4-entry, fully-associative, FIFO
Page size	4 KB (12-bit offset)
Instruction memory	16 KB BRAM
Data memory	16 KB BRAM
Register file	32 × 32-bit
Target FPGA	xc7z010clg400-1 (Zybo Z7-10)
CPU clock	62.5 MHz (divided from 125 MHz)

III. CSR INFRASTRUCTURE

The processor implements 10 CSR registers essential for trap handling and TLB management:

- **Standard M-mode CSRs:** MSTATUS (0x300), MTVEC (0x305), MSCRATCH (0x340), MEPC (0x341), MCAUSE (0x342), MTVAL (0x343), SATP (0x180)
- **Custom TLB CSRs:** TLB_VPN (0xFC0), TLB_PPN (0xFC1), TLB_CMD (0xFC2)

CSR reads are combinational in the EX stage; writes are sequential in the MEM stage. Trap entry has priority over normal CSR writes, ensuring atomic save of MEPC, MCAUSE, and MTVAL.

The three custom CSRs provide a software interface for TLB management: the trap handler stages a VPN and PPN, then writes TLB_CMD=1 to commit the entry or TLB_CMD=2 to flush all entries.

IV. TLB DESIGN AND VIRTUAL MEMORY

A. TLB Structure

The TLB is a 4-entry fully-associative cache. Each entry stores only three fields: a valid bit, a 20-bit Virtual Page Number (VPN), and a 22-bit Physical Page Number (PPN)—totaling 43 bits per entry. Lookup is combinational; all four entries are compared in parallel against the requested VPN.

Note on permission bits: Unlike a full Sv32 TLB, our implementation does *not* store or enforce R/W/X/U permission bits from the page table entries. All valid TLB hits allow both read and write access. This is a deliberate simplification—permission fault checking (RISC-V cause codes 5 and 7) is deferred to a future enhancement.

B. FIFO Replacement Policy

New entries are written at the position indicated by a 2-bit `next_write_index` counter, which increments on each write and wraps around (0 → 1 → 2 → 3 → 0). This provides deterministic, hardware-simple replacement.

Justification: FIFO was chosen over LRU for three reasons: (1) *hardware cost*—FIFO requires only a 2-bit counter (2 flip-flops), while LRU for 4 entries needs 4 × 2-bit age counters plus priority logic (~10× more hardware); (2) *benchmark equivalence*—for our sequential scan workloads, FIFO and LRU produce identical replacement decisions; and (3) *research focus*—the TLB capacity cliff (the primary phenomenon under study) occurs identically under both policies when the working set exceeds TLB capacity.

C. Translation Enable Conditions

Address translation is active when `satp ≠ 0 AND !handler_mode`. The `handler_mode` flag is set on trap entry and cleared on MRET, allowing the trap handler to access physical memory directly (identity-mapped) for page table walks.

SATP simplification: In the standard RISC-V Sv32 specification, SATP contains a MODE field (bit 31), an ASID field, and a 22-bit PPN. Our implementation uses SATP purely as a 32-bit base address register pointing to the root page table. Translation is enabled whenever SATP ≠ 0. There is no MODE bit, ASID, or privilege-mode gating. This is valid because the processor operates exclusively in M-mode.

D. Page Fault Flow

On a TLB miss:

- 1) MEM stage detects miss, generates trap (cause 13 for loads, 15 for stores)
- 2) Pipeline flushes; MEPC, MCAUSE, MTVAL saved to CSRs
- 3) PC redirects to MTVEC (trap handler entry point)
- 4) Software handler reads MTVAL, walks the 2-level Sv32 page table in data memory
- 5) Handler writes TLB entry via custom CSRs (TLB_VPN, TLB_PPN, TLB_CMD)
- 6) MRET returns to MEPC, replaying the faulting instruction

- 7) Replayed instruction hits in TLB and completes normally

The measured miss penalty is 46 cycles per fault, comprising pipeline flush (4 cycles), handler execution (36 instructions + 2 load-use stalls), MRET pipeline flush (4 cycles), and instruction replay.

V. FPGA IMPLEMENTATION

A. System Architecture

The FPGA top module (`fpga_top`) instantiates a clock divider (125 MHz $\rightarrow$ 62.5 MHz) and the microprocessor module. The clock divider ensures timing closure on the xc7z010's fabric.

B. Debug Infrastructure: ILA

Performance data is captured via a Xilinx Integrated Logic Analyzer (ILA) debug core with 15 probes (269 bits total) and 1024-sample depth. This replaced an earlier UART-based reporting interface (`perf_reporter + uart_tx`), eliminating the need for external USB-UART cables.

The ILA captures all six performance counters (total cycles, stall cycles, instruction count, TLB accesses/hits/misses), the program counter, data memory address, pipeline valid bits, real-time TLB signals, and CPU status. The primary trigger is the rising edge of `cpu_halted`, with the trigger position set to mid-buffer (512 samples pre/post) for comprehensive capture around the halt event.

The ILA consumes approximately 8 BRAMs (6.7% of the Z7-10's 120 BRAMs), a modest resource cost that enables powerful non-intrusive debugging.

C. LED Status Indicators

Four LEDs provide real-time status: LED0 indicates CPU halt, LED1 signals trap events, LED2 pulses on TLB misses, and LED3 provides a heartbeat from the cycle counter.

VI. VERIFICATION

The design was verified through four progressively complex testbenches, each targeting a specific verification phase.

A. Phase 0: RV32I Pipeline Regression

A 49-instruction test program exercises all RV32I instructions through the pipeline, including forwarding paths, load-use stalls, and branch flushes. All 32 registers were verified against expected values.

Results: 53 cycles, 39 retired instructions, 1 stall cycle, CPI = 1.36. **32/32 register tests passed.**

B. Phase 1: CSR Infrastructure

A 70-instruction test verifies CSRRW/CSRRS/CSRRC/C-SRRWI operations and the complete ECALL $\rightarrow$ MTVEC $\rightarrow$ handler $\rightarrow$ MRET $\rightarrow$ continue flow.

Results: 9/9 tests passed.

C. Phase 2–3: VM/TLB Full-Flow

An 85-instruction test exercises end-to-end virtual memory: TLB lookups, page fault traps, 2-level page table walks, TLB refill, TLB flush (SFENCE), and instruction replay for both loads and stores.

Results: 16 TLB accesses, 9 hits, 7 misses (6 load faults + 1 store fault). **All 16/16 checks passed.**

D. Phase 4–5: TLB Performance Benchmarks

Four test configurations with varying working-set sizes characterize TLB performance. Each test uses the same benchmark program with different page counts (Table II).

TABLE II
TLB PERFORMANCE RESULTS (SIMULATION)

Test	Pages	Accesses	Hits	Misses	Hit Rate
T1	1	11	10	1	90%
T2	4	44	40	4	90%
T3	8	160	80	80	50%
T4	16	320	160	160	50%

E. FPGA Deployment

The design was successfully synthesized, implemented, and deployed on the Zybo Z7-10 FPGA at 62.5 MHz. Performance data was captured via the ILA debug core in Vivado Hardware Manager. The ILA waveforms confirmed the expected counter values, validating the FPGA implementation against simulation results.

VII. PERFORMANCE ANALYSIS

A. Experimental Methodology

Performance data was collected from **FPGA hardware** (Zybo Z7-10 at 62.5 MHz) using the Xilinx ILA debug core, which captures cycle-accurate counter values at CPU halt. The benchmark program (`t_perf_instr.mem`) is **identical** across all four tests—only the data memory initialization file changes (varying page counts), ensuring the only independent variable is working-set size. Each test was run by re-programming the bitstream. Results are deterministic: identical bitstreams produce identical counter values. The memory model uses single-cycle BRAM with no cache hierarchy.

B. TLB Capacity Cliff

The results reveal a dramatic performance cliff at the TLB capacity boundary (4 entries):

- **T1/T2 (≤ 4 pages):** 90% hit rate. Only cold-start misses occur.
- **T3/T4 (> 4 pages):** 50% *observed* hit rate, but the **logical miss rate is 100%**. This observed 50% is **misleading as a performance indicator**: it arises from instruction replay, where each miss triggers a handler that refills the TLB, then the faulting instruction re-executes and hits, producing $2N$ observed accesses from N logical misses. All subsequent analysis uses the *logical* miss rate.

C. Miss Penalty Analysis

The trap handler at PC $0 \times 200 \text{--} 0 \times 28C$ performs a 2-level software page table walk in 36 instructions: context save (6), address parsing (5), L1 walk (5), L2 walk (6), TLB fill (4), context restore (6), and MRET (1). Including pipeline flush penalties (4 cycles entry + 4 cycles MRET) and 2 load-use stalls, the analytical per-miss cost is **46 cycles**.

Cross-validation via the $T3 \leftrightarrow T4$ differential yields $(8215 - 4135)/(160 - 80) = 51$ cycles, slightly higher because it includes additional baseline loop iterations.

D. Formal Definitions

Definition (Address-Translation Overhead). The fraction of total execution time attributable to address translation:

$$\text{Overhead} = \frac{C_{\text{vm}} - C_{\text{baseline}}}{C_{\text{vm}}} \times 100\% \quad (1)$$

where C_{vm} is the measured cycle count with translation enabled, and C_{baseline} is the cycle count for the identical workload on a base CPU without translation. Normalization is by C_{vm} (not C_{baseline}) because we measure what fraction of the *actual* execution is overhead, not how much slower the system became.

The baseline was derived from **the original base RV32I CPU** (before TLB/CSR integration), running the identical loop structure (10 outer iterations $\times$ N inner data accesses). Both CPUs share the same 5-stage pipeline, forwarding paths, and 2-cycle taken-branch penalty, ensuring the comparison isolates TLB overhead. The formula $C_{\text{baseline}} = 60N + 38$ was verified by pipeline trace analysis and cross-checked against the base CPU’s performance counters.

E. Results

TABLE III
COMPLETE PERFORMANCE RESULTS FROM FPGA ILA

Test	Pg	Cycles	Instrs	Misses	CPI	Hit%	Overhead%
T1	1	160	128	1	1.25	90.9	38.8
T2	4	475	356	4	1.33	90.9	41.5
T3	8	4,135	3,252	80	1.27	50.0	87.5
T4	16	8,215	6,452	160	1.27	50.0	87.9

Instruction counts were verified analytically: $\text{total} = \text{main_instrs} + (\text{misses} \times 36)$, matching all four ILA captures exactly. The CPI values (1.25–1.33) appear similar because the instruction counter includes handler instructions; the meaningful metric is overhead percentage.

Memory model caveat: These results assume **ideal single-cycle BRAM**. In a real system with DRAM, page table walk loads would incur cache miss latency (50–200 cycles), inflating miss penalty from 46 to 200–500+ cycles. The *relative* overhead trend remains valid regardless of memory technology.

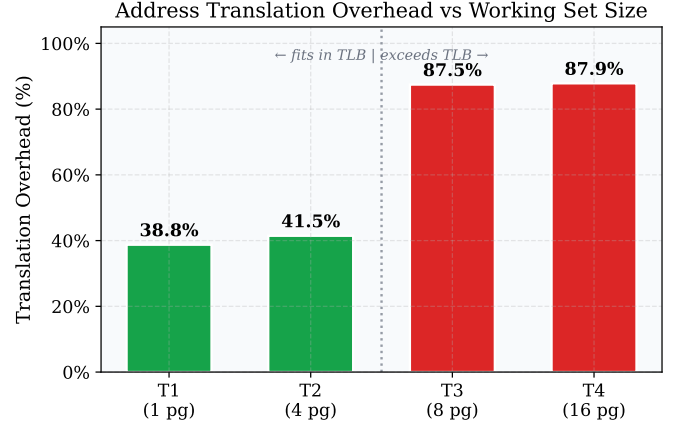


Fig. 1. Address translation overhead as a fraction of total execution time. The TLB capacity cliff is visible: overhead jumps from 39–42% (fits in TLB) to 87–88% (exceeds TLB).

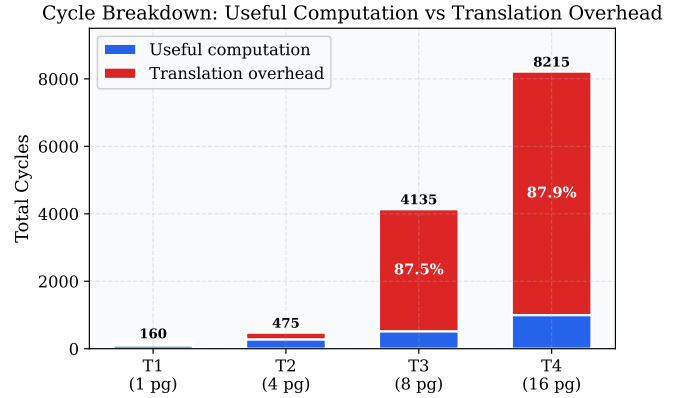


Fig. 2. Cycle breakdown showing useful computation (blue) vs translation overhead (red). Under thrashing (T3/T4), the processor spends nearly all cycles executing the trap handler.

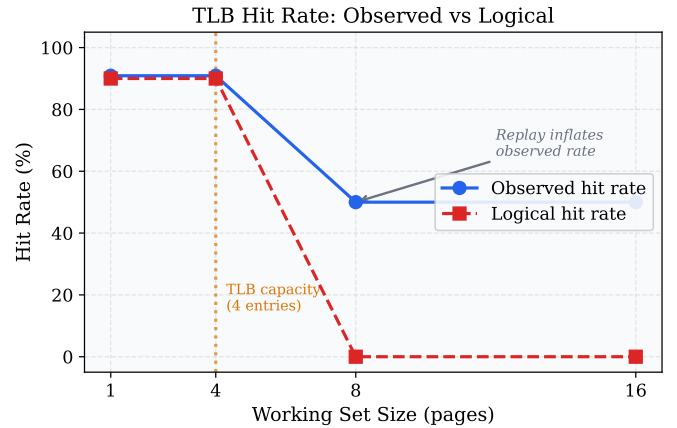


Fig. 3. TLB hit rate: observed vs logical. The observed 50% under thrashing is misleading—it arises from instruction replay. The logical miss rate is 100%.

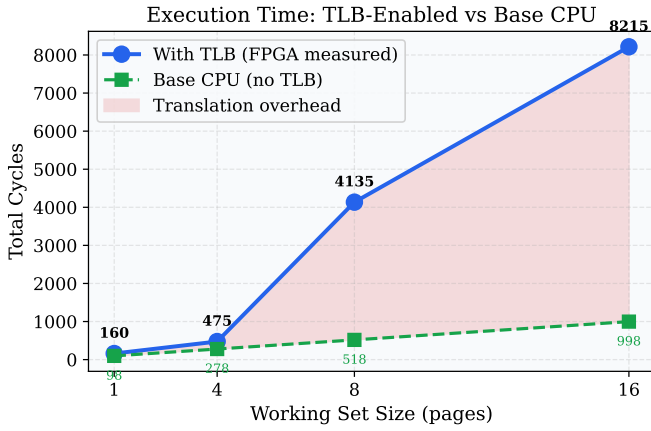


Fig. 4. Total execution cycles with VM enabled (measured via FPGA ILA) vs base CPU without TLB. The shaded region represents the translation overhead. Both CPUs share identical pipeline and branch penalty.

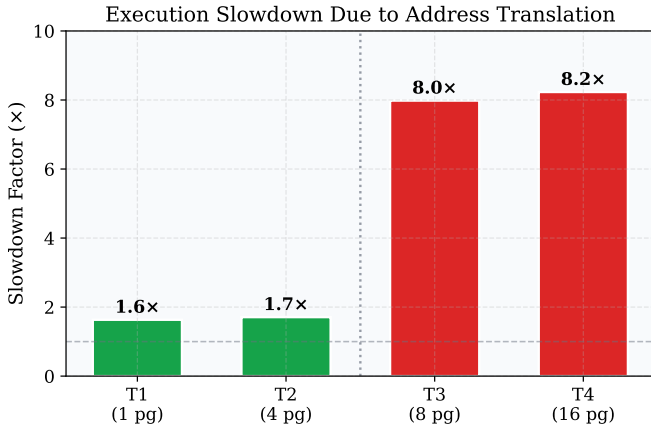


Fig. 5. Execution slowdown factor relative to the base CPU. Under thrashing (T3/T4), the TLB-enabled CPU is $\sim 8\times$ slower.

F. EMAT Analysis

For theoretical completeness, the Effective Memory Access Time provides a standard metric:

$$\text{EMAT} = h \times t_{\text{hit}} + (1 - h) \times (t_{\text{hit}} + t_{\text{miss}}) \quad (2)$$

where h is the *logical* hit rate (not the observed rate inflated by replays), $t_{\text{hit}} = 1$ cycle, and $t_{\text{miss}} = 46$ cycles. For T1/T2 ($h = 0.909$): $\text{EMAT} = 0.909 + 0.091 \times 47 = 5.2$ cycles. For T3/T4 ($h = 0$, logical): $\text{EMAT} = 47$ cycles. Note that EMAT is a *theoretical reference* assuming a simple cache-like model; the primary performance metric in this work is the directly measured overhead percentage, which captures the full system behavior including pipeline flushes and instruction replay.

G. Answer to Research Question

When the working set fits in the TLB, overhead is 38.8% (T1) to 41.5% (T2)—only cold-start misses, which would be amortized in longer programs. When TLB capacity is exceeded, overhead jumps to **87–88%**: 87.5% for 8 pages

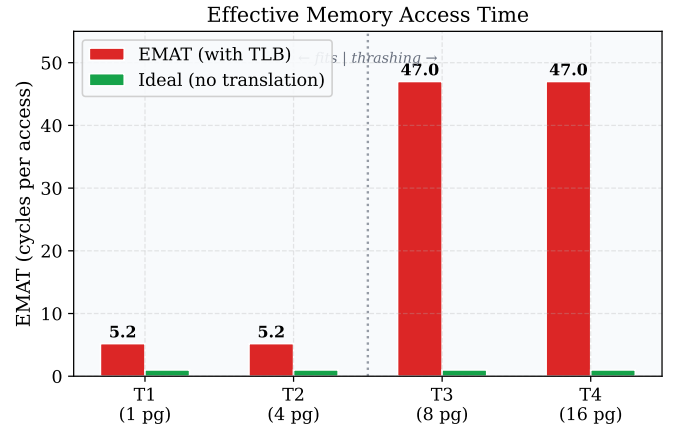


Fig. 6. Effective Memory Access Time: 5.2 cycles when fitting in TLB vs 47 cycles under thrashing ($47\times$ the ideal single-cycle access).

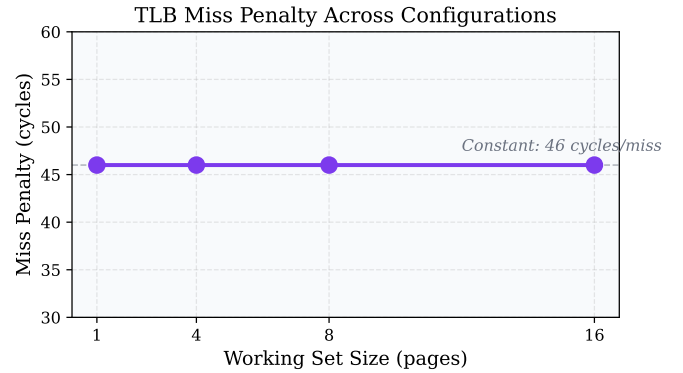


Fig. 7. TLB miss penalty remains constant at 46 cycles across all configurations, confirming deterministic handler execution independent of working-set size.

($2\times$ TLB) and 87.9% for 16 pages ($4\times$ TLB). The TLB-enabled CPU is $\sim 8\times$ slower than the base CPU under thrashing. Each miss costs 46 cycles, and in the thrashing regime the processor spends nearly all its time in the trap handler. This confirms that TLB sizing relative to working-set size is the single most critical determinant of memory system performance in a software-managed TLB architecture.

VIII. DESIGN EVOLUTION

The architecture was developed in five phases:

- 1) **Phase 0:** Base 5-stage pipelined RV32I processor with forwarding, hazard detection, and full ISA verification.
- 2) **Phase 1:** CSR infrastructure (MSTATUS, MTVEC, MEPC, MCAUSE, MSCRATCH, SATP) and ECAL-L/MRET trap flow.
- 3) **Phase 2–3:** 4-entry TLB with FIFO replacement, page fault handling, software trap handler, and handler_mode for identity-mapped page table walks.
- 4) **Phase 4:** FPGA deployment with clock divider, initially UART-based performance reporting (later replaced by ILA).

- 5) **Phase 5:** Performance benchmarking with 4-test suite across varying working-set sizes.

A significant infrastructure change was the replacement of the UART-based performance reporting system (`perf_reporter.v` + `uart_tx.v`, ~286 lines) with a Xilinx ILA debug core. This simplified the design by removing the need for external USB-UART hardware and serial terminal software, while providing richer real-time debugging capabilities including pipeline state visibility and advanced triggering.

IX. RESOURCE UTILIZATION

TABLE IV
FPGA RESOURCE SUMMARY (POST-PLACE-AND-ROUTE)

Resource	Used	Available	Util.
Slice LUTs	9,395	17,600	53.4%
LUT as Logic	5,978	17,600	34.0%
LUT as Distributed RAM	2,324	6,000	38.7%
LUT as Shift Register	1,093	6,000	18.2%
Slice Registers	10,055	35,200	28.6%
Slices	3,797	4,400	86.3%
BRAM (36K+18K)	7.5	60	12.5%
DSP48	0	80	0%
Bonded IOBs	6	100	6.0%

Timing closure: All timing constraints were met. The worst negative slack (WNS) is +1.206 ns on the `cpu_clk` domain (16 ns period, 62.5 MHz), indicating a critical path delay of 14.794 ns. Hold slack is +0.015 ns with zero failing endpoints across 42,509 total endpoints.

Power: Total on-chip power is 0.137 W (43 mW dynamic + 94 mW static). The ILA debug core consumes 25 mW (58% of dynamic power); the CPU core itself uses only 11 mW. A production deployment without the ILA would reduce total power to approximately 0.112 W.

The design uses a modest fraction of the Z7-10's resources. The ILA debug core accounts for the majority of BRAM usage and register count increase from synthesis to implementation. Slice utilization at 86.3% is the tightest resource constraint.

X. LIMITATIONS AND FUTURE WORK

A. Current Limitations

- Static not-taken branch prediction incurs a 2-cycle penalty on every taken branch (IF/ID and ID/EX flushed).
- FIFO replacement does not exploit temporal locality; LRU would benefit workloads with hot/cold access patterns.
- Software page table walk (46 cycles measured) is significantly slower than a hardware walker (~2–5 cycles).
- No instruction TLB—only data memory accesses are translated.
- Single-cycle memory model does not reflect realistic cache miss latencies.

B. Potential Enhancements

- LRU replacement policy (parameterized via `USE_LRU` flag).

- Hardware page table walker for reduced miss penalty.
- Permission checking (R/W/X bits from PTE).
- Separate instruction TLB for fetch stage translation.
- Configurable TLB size (4/8/16 entries) for comparative analysis.
- Hot-cold access pattern benchmarks to demonstrate LRU advantage.

XI. CONCLUSION

We have designed, verified, and deployed a complete RISC-V RV32I pipelined processor with software-managed Sv32-style virtual memory on the Zybo Z7-10 FPGA. The empirical performance analysis demonstrates that:

- 1) The 4-entry FIFO TLB achieves 90.9% observed hit rates when the working set fits within TLB capacity, with address-translation overhead of 39–42% (cold-start misses that would amortize in longer programs).
- 2) A sharp capacity cliff occurs when the working set exceeds 4 pages: the logical miss rate reaches 100%, and address-translation overhead jumps to **87–88%** of total execution time, representing an 8× slowdown versus the base CPU.
- 3) Each TLB miss costs 46 cycles (pipeline flush, 36-instruction software handler, MRET, and instruction replay), as verified by consistent instruction count decomposition across all four FPGA captures.
- 4) The ILA-based debug infrastructure provides efficient, non-intrusive performance data capture on FPGA hardware.

The complete design comprises approximately 1,975 lines of synthesizable Verilog across 24 source modules, with 4 testbenches totaling ~798 lines providing comprehensive verification coverage from individual instruction correctness to system-level virtual memory flows.

ACKNOWLEDGMENT

This project was developed as part of the Computer Organization and Architecture course. The RISC-V ISA specification [1] and the RISC-V Privileged Architecture specification [2] provided the architectural foundation.

REFERENCES

- [1] A. Waterman and K. Asanović, Eds., “The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA,” Document Version 20191213, RISC-V Foundation, Dec. 2019.
- [2] A. Waterman and K. Asanović, Eds., “The RISC-V Instruction Set Manual, Volume II: Privileged Architecture,” Document Version 20211203, RISC-V Foundation, Dec. 2021.
- [3] J. L. Hennessy and D. A. Patterson, *Computer Architecture: A Quantitative Approach*, 6th ed. Cambridge, MA: Morgan Kaufmann, 2019.
- [4] D. A. Patterson and J. L. Hennessy, *Computer Organization and Design RISC-V Edition: The Hardware/Software Interface*, 2nd ed. Cambridge, MA: Morgan Kaufmann, 2021.
- [5] Xilinx, “Vivado Design Suite User Guide: Programming and Debugging,” UG908, v2024.2, 2024.
- [6] Digilent, “Zybo Z7 Reference Manual,” Rev. B, 2023. [Online]. Available: <https://digilent.com/reference/programmable-logic/zybo-z7/reference-manual>